

Module 6: Lesson 3

Further autoencoder topics



Outline

- ▶ Sparse autoencoders.
- ▶ Setting constraints on autoencoders.
- ▶ Autoencoders for images.

Sparse autoencoders

Sparse autoencoders represent a type of autoencoder where a penalty term is imposed on the realizations of the encodings.

This type of penalization may be of interest in overcomplete autoencoders where the number of encodings in the hidden layers is greater than the number of features.

In a sparse autoencoder, we add an L1-penalty term on the encodings Z .

If we use the MSE as the performance metric, the loss function of a sparse autoencoder can be as follows:

$$\left\| X - \hat{X} \right\|_F^2 + \lambda_1 \| \text{vec}(Z) \|_1$$

$\text{vec}(\cdot)$ is the operator that transforms a matrix into a column vector and $\| \cdot \|_1$ is the L1-norm.

In TensorFlow, we can add an activity regularizer to the layer that leads to the bottleneck:

```
activity_regularizer= tf.keras.regularizers.l1(l1weight)
```

where **l1weight** is the λ_1 hyperparameter.

Setting constraints on autoencoders

In Lesson 1, we described the architecture of an autoencoder and its association with PCA.

A single-layer linear autoencoder performs the same task as PCA, but with fewer constraints on the architecture.

Such a lack of constraints may provide too much leeway and lead to ill-posed models.

- ▶ Usual outcomes of such flexibility are the overfitting and exploding/vanishing gradient problems.
- ▶ To avoid those issues we may opt for excessively deep and complex networks, losing interpretability.

We can add the constraints from PCA to discipline the behavior of autoencoders. Namely, in PCA:

- ▶ The weight matrix is orthonormal (orthogonality + unit norm), $W^T W = I_{k \times k}$.
- ▶ The matrix of decoding weights is the transpose matrix of encoding weights, $W = W_2 = W_1^T$.
- ▶ The encodings are uncorrelated, $\text{Cov}(Z_k)$ is a diagonal matrix.

We are going to exploit TensorFlow-Keras capabilities to include constraints and loss terms in the layers of autoencoders as tools that may alleviate the under-performance of an unconstrained autoencoder.

Constraints: Unit norm weights

In PCA, the eigenvectors are constrained to have a unit norm.

The elements of the weight matrices may grow arbitrarily large with the norm of its vectors, leading to arbitrarily large variances.

- ▶ A unit-norm constraint may solve exploding-gradient problems.

TensorFlow provides a built-in method to constrain the weights to have a unit norm:

```
kernel_constraint = tf.keras.constraints.UnitNorm(axis=axis)
```

Remember that, in PCA, the encoding layer is computed as $Z_k = P_k^T X_0$, meaning that the unit norm must be imposed over the rows of the weights of the encoding layers (**axis=0**).

Thus, in the decoding layers, we must impose the unit norm over the columns in the corresponding weight matrix (**axis=1**).

- ▶ The same applies to the orthogonality constraint that we introduce next.

Constraints: Orthogonal weights

Weight orthogonality implies that weights transmit valuable information that is not shared across layers.

- An orthogonality constraint may solve vanishing-gradient problems.

Orthogonality imposes the inner products of the decoding vectors to be zero:

$$w_i^T w_j = 0, \text{ for } i, j = 1, \dots, k \text{ and } i \neq j$$

Via subclassing, we can build a custom kernel regularizer that penalizes the sum of the inner products:

kernel_regularizer=WeightsOrthogonalityRegularizer(encoding_dim, weight, axis=axis)

```
class WeightsOrthogonalityRegularizer(tf.keras.regularizers.Regularizer):
    def __init__(self, encoding_dim, weightage = 1.0, axis = 0):
        self.encoding_dim = encoding_dim
        self.weightage = weightage
        self.axis = axis
    def weights_orthogonality(self, w):
        if(self.axis==1):
            w = tf.transpose(w)
            m = tf.tensordot(tf.transpose(w), w, axes=1) - tf.math.multiply(w,tf.eye(self.encoding_dim))
            return self.weightage * tf.math.sqrt(tf.math.reduce_sum(tf.square(m)))
        def __call__(self, w):
            return self.weights_orthogonality(w)
```

Constraints: Tied weights

Another manner of constraining and regularizing the autoencoder is to tie weights across layers (we considerably reduce the number of trainable parameters).

This means setting the weight matrix in layer ℓ equal to the transpose weight matrix in layer $-\ell$ (ℓ layers before the output layer).

To implement this, we need to define a new type of layer in TensorFlow:

`decoder_l = DenseTranspose(encoder_l, activation)`
`encoder_l` refers to the layer that `decoder_l` has tied weights with.

```
class DenseTranspose(tf.keras.layers.Layer):
    def __init__(self, dense, activation=None):
        self.dense = dense
        self.activation = tf.keras.activations.get(activation)
    def build(self, batch_input_shape):
        self.biases = self.add_weight(name="bias", shape=[self.dense.input_shape[-1]], initializer="zeros")
        super().build(batch_input_shape)
    def call(self, inputs):
        z = tf.matmul(inputs, self.dense.weights[0], transpose_b=True)
        return self.activation(z + self.biases)
```

Constraints: Uncorrelated encodings

When the encodings (factors) are correlated, redundant information is transferred through the network.

We can also impose the encodings to be uncorrelated through an activity regularizer.

```
activity_regularizer = UncorrelatedFeaturesConstraint(encoding_dim, weightage)
```

```
class UncorrelatedFeaturesConstraint(tf.keras.constraints.Constraint):
    def __init__(self, encoding_dim, weightage = 0.0):
        self.encoding_dim = encoding_dim
        self.weightage = weightage
    def uncorrelated_feature(self, x):
        covariance = tfp.stats.covariance(x)
        m = tf.square(covariance - tf.math.multiply(covariance, tf.eye(self.encoding_dim)))
        output = self.weightage * tf.math.sqrt(tf.math.reduce_sum(m))
        return output
    def __call__(self, x):
        return self.uncorrelated_feature(x)
```

Autoencoders for images: Dense layers

Image feature extraction and recognition are the main applications of deep learning architectures. This also applies to autoencoders.

We can easily build autoencoders for images with dense layers by including a flattening layer as the input layer and a reshaping layer as the output layer.

```
encoder = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=[window_size, window_size]),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu"),
])
decoder = tf.keras.models.Sequential([
    tf.keras.layers.Dense(100, activation="relu", input_shape=[30]),
    tf.keras.layers.Dense(window_size * window_size, activation="sigmoid"),
    tf.keras.layers.Reshape([window_size, window_size])
])
autoencoder = tf.keras.models.Sequential([encoder, decoder])
adam = tf.keras.optimizers.Adam(learning_rate=1e-4)
autoencoder.compile(loss="mse", optimizer=adam)
autoencoder.fit(X_data_train, X_data_train, epochs=20, validation_data=(X_data_test, X_data_test))
```

Autoencoders for images: Convolutional layers

We can also build autoencoders with convolutional layers, which tend to reduce the number of parameters of the architecture relative to dense layers.

The layers of the decoder must be “Transpose convolutions” that perform the reverse operations of the convolutions that we studied in Modules 2 and 3.

```
conv_encoder = tf.keras.models.Sequential([
    tf.keras.layers.Reshape([window_size, window_size, 1], input_shape=[window_size, window_size]),
    tf.keras.layers.Conv2D(16, kernel_size=3, padding="SAME", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),
    tf.keras.layers.Conv2D(32, kernel_size=3, padding="SAME", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),
    tf.keras.layers.Conv2D(64, kernel_size=3, padding="SAME", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2)
])
conv_decoder = tf.keras.models.Sequential([
    tf.keras.layers.Conv2DTranspose(32, kernel_size=3, strides=2, padding="VALID", activation="relu", input_shape=[3, 3, 64]),
    tf.keras.layers.Conv2DTranspose(16, kernel_size=3, strides=2, padding="SAME", activation="relu"),
    tf.keras.layers.Conv2DTranspose(1, kernel_size=3, strides=2, padding="SAME", activation="sigmoid"),
    tf.keras.layers.Reshape([window_size, window_size])
])
conv_ae = tf.keras.models.Sequential([conv_encoder, conv_decoder])
```

Summary of Lesson 3

In Lesson 3, we have:

- ▶ Described the concept of sparse autoencoder.
- ▶ Enumerated several tools to regularize autoencoders based on PCA.
- ▶ Outlined the use of autoencoders for image data.

⇒ **References for this lesson:**

Ranjan, C. "Build the Right Autoencoder — Tune and Optimize using PCA Principles (Parts I and II)." *Medium*. July 2019.

Geron, A. "Machine Learning Notebooks: Autoencoders and GANS". *Github*. Mar 2020).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will train a sparse autoencoder and implement it for portfolio choice.

See you there!